

Digital Saarthi

Architecture Documentation

AI-Powered Action-Guidance Layer for Government Services

Version 1.0 | September 2026 | Comprehensive Technical Guide

Digital Saarthi - Comprehensive Architecture Documentation

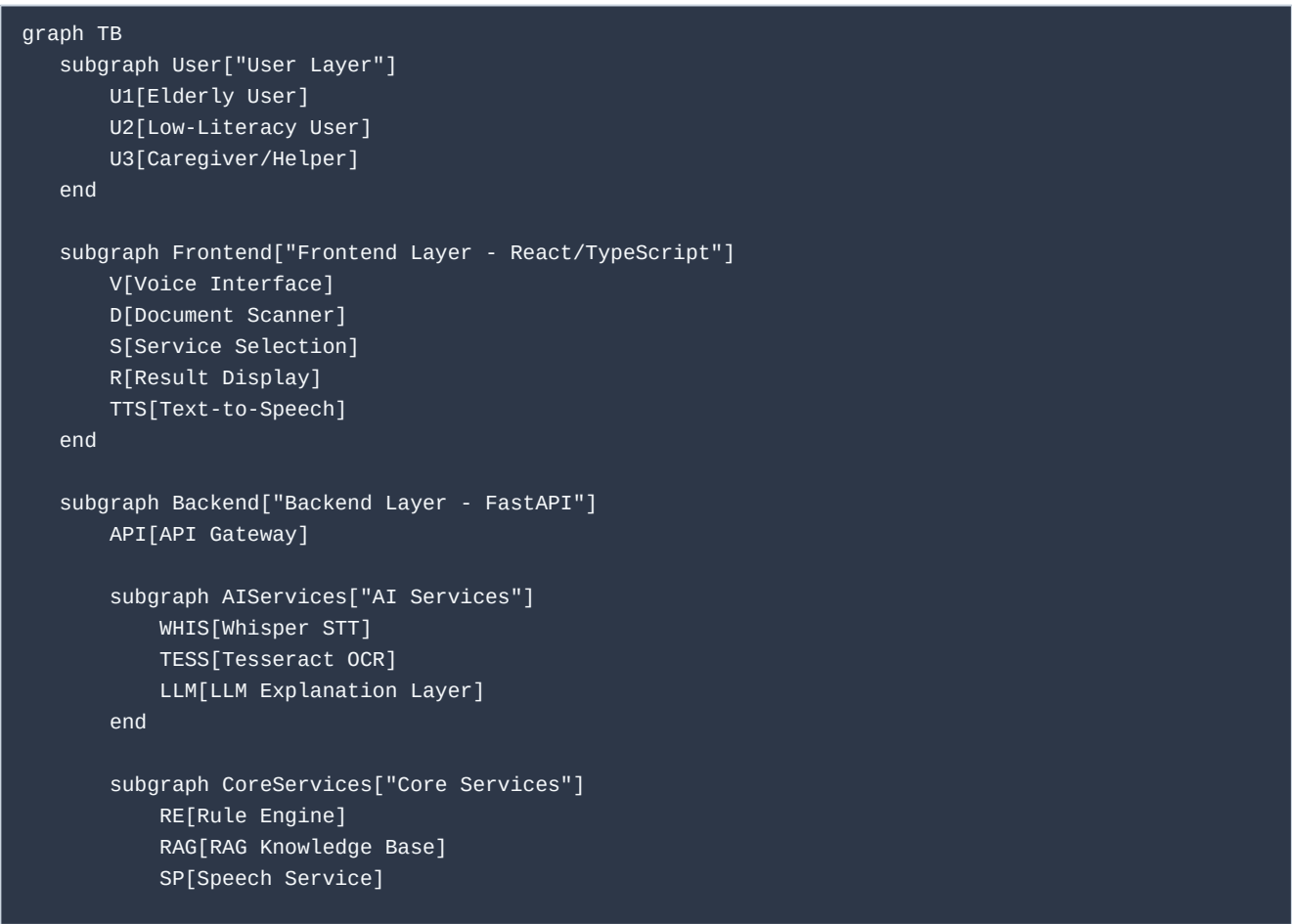
1. Executive Summary

Digital Saarthi is an AI-powered "Action-Guidance Layer" for government services that helps elderly and low-literacy users access public services like pensions and welfare schemes. It serves as a bridge between complex government processes and underserved communities who struggle with digital interfaces.

Key Value Propositions

- **Accessibility First:** Voice interface in Hindi/English for users unfamiliar with technology
- **Verified Guidance:** Hardcoded rules and verified knowledge base prevent harmful hallucinations
- **Step-by-Step Action Plans:** Converts complex procedures into simple, actionable steps
- **Document Intelligence:** OCR-powered extraction of information from government documents
- **Privacy-Focused:** Local processing options and no data retention for public model training

2. System Architecture Diagram



```
end
end

subgraph Data["Data Layer"]
  KB[Government Schemes Knowledge Base]
  Rules[Eligibility Rules DB]
  Cache[Response Cache]
end

U1 -->|Voice/Text| V
U1 -->|Document Upload| D
U2 -->|Voice/Text| V
U3 -->|Help| D

V --> Frontend
D --> Frontend

Frontend -->|HTTP/JSON| API

API --> WHIS
API --> TESS
API --> RE
API --> RAG
API --> SP

WHIS --> LLM
TESS --> RE
RE --> RAG
RAG --> LLM

RE --> Rules
RAG --> KB

LLM -->|Simple Language Response| Frontend
RE -->|Eligibility Decision| Frontend
RAG -->|Verified Info| Frontend

style User fill:#e1f5fe
style Frontend fill:#e8f5e9
style Backend fill:#fff3e0
style AIServices fill:#fce4ec
style CoreServices fill:#f3e5f5
style Data fill:#fffde7
```

Architecture Layers Explained

Layer	Technology	Purpose
User Interface	React 18 + TypeScript	Accessibility-first web interface
API Gateway	FastAPI	REST API with automatic OpenAPI docs
Speech Processing	OpenAI Whisper	Speech-to-text for voice queries

Layer	Technology	Purpose
Document Processing	Tesseract OCR	Text extraction from images
Knowledge Base	RAG System	Verified government schemes data
Decision Engine	Rule Engine	Hardcoded eligibility verification
Explanation Layer	LLM (Optional)	Translate complex info to simple language

3. Component Breakdown

3.1 Frontend (React/TypeScript)

Location: frontend/

Key Technologies: - React 18.2.0 - TypeScript 4.9.5 - CSS for styling

Components:

```
graph LR
  App[App.tsx<br/>Main Container] --> VI[VoiceInterface.tsx]
  App --> DC[DocUpload.tsx]
  App --> RC[ResultCard.tsx]

  VI -->|"onQuery()"| API[Backend API]
  DC -->|"onUpload()"| API
  API -->|"eligibilityResult"| RC
```

How It Works:

- 1. **VoiceInterface.tsx:** Provides microphone button for voice input
- 2. Accepts Hindi/English speech input
- 3. Sends text queries to backend
- 4. Displays text-to-speech responses
- 5. **DocUpload.tsx:** Document scanning interface
- 6. File upload for Aadhaar cards
- 7. Demo mode toggle for hackathons
- 8. Shows extracted data preview
- 9. **ResultCard.tsx:** Displays eligibility results
- 10. Shows verdict (Eligible/Not Eligible)
- 11. Lists reasons for decision

12. Provides step-by-step action plan

Accessibility Features: - Large touch targets (minimum 64px height) - High contrast colors for visibility - Text-to-speech support for blind users - Keyboard navigation support - Screen reader friendly components

3.2 Backend (FastAPI)

Location: backend/app/

Key Technologies: - FastAPI 0.111.0 - Uvicorn (ASGI server) - Pydantic (data validation) - Pillow + pytesseract (OCR)

API Endpoints:

```
graph TB
    subgraph Endpoints["API Endpoints"]
        R1["GET /<br/>Root Info"]
        R2["POST /api/voice-query<br/>Process voice/text"]
        R3["POST /api/scan-document<br/>Extract from document"]
        R4["POST /api/check-eligibility<br/>Verify eligibility"]
        R5["GET /api/schemes<br/>Search schemes"]
        R6["GET /api/schemes/{id}<br/>Get scheme details"]
        R7["GET /api/health<br/>Health check"]
    end
```

Endpoint	Method	Purpose
/	GET	API information
/api/voice-query	POST	Process voice/text input
/api/scan-document	POST	Extract data from documents
/api/check-eligibility	POST	Check scheme eligibility
/api/schemes	GET	Search government schemes
/api/schemes/{scheme_id}	GET	Get scheme details
/api/health	GET	Health check

3.3 AI/ML Components

3.3.1 Whisper (Speech-to-Text)

Implementation: backend/app/speech_service.py

How It Works: 1. User speaks in Hindi or English (or code-switches) 2. Audio is sent to the speech service 3. Intent detection identifies the user's goal 4. Returns appropriate response with suggested actions

Supported Intents: - `pension_inquiry` : Questions about old age pension - `eshram_inquiry` : Questions about worker registration - `document_help` : Help with documents - `general_help` : General assistance

Code-Switching Support: Handles mixed Hindi-English text common in India

```
# Example intent detection logic
pension_keywords = [ 'pension', 'old age', 'old age pension', 'ignaps' ]
any_keyword_in_query = any(keyword in query for keyword in pension_keywords)
intent = 'pension_inquiry'
confidence = 0.9
```

3.3.2 Tesseract OCR

Implementation: `backend/app/ocr_service.py`

How It Works: 1. User uploads image (Aadhaar card, government letter) 2. Tesseract extracts raw text from image 3. Regex patterns extract structured data: - Name - Date of Birth - Age - Gender - BPL status

Extracted Fields:

```
{
  'name': 'Kamkaji Devi',
  'dob': '12/03/1954',
  'age': 70,
  'gender': 'Female',
  'has_aadhaar': True
}
```

Fallback: When Tesseract is unavailable, demo mode provides realistic sample data

3.3.3 LLM (Explanation Layer)

Purpose: - Translates complex government procedures into simple language - NEVER makes eligibility decisions (this is the Rule Engine's job) - Explains verified outputs from RAG and Rule Engine - Provides guided responses in Hindi/English

Important Safety Constraint:

The LLM only *explains* verified facts. It does not determine eligibility. This prevents hallucinations that could mislead vulnerable users about their benefits.

3.3.4 RAG (Retrieval Augmented Generation)

Implementation: `backend/app/rag_service.py`

How It Works: 1. Contains verified knowledge base of government schemes 2. Source: official government websites (nsap.nic.in, eshram.gov.in, pmkisan.gov.in) 3. Search functionality retrieves relevant scheme information 4. Returns structured data including: - Eligibility criteria - Required documents - Application process - Benefits - Helpline numbers

Supported Schemes:

Scheme	Full Name	Category
IGNOAPS	Indira Gandhi National Old Age Pension	Social Security
E-Shram	E-Shram Portal for Unorganised Workers	Labor & Employment
PM-Kisan	Pradhan Mantri Kisan Samman Nidhi	Agriculture

3.3.5 Rule Engine (Safety Core)

Implementation: backend/app/rule_engine.py

How It Works: - Hardcoded, auditable eligibility rules - No AI/LLM involved in decision-making - Returns structured EligibilityResult with: - **eligible**: Boolean decision - **verdict**: Short label for display - **reasons**: Human-readable explanations - **steps**: Action roadmap

IGNOAPS Rules:

```
Age >= 60 AND BPL Card = Required
```

E-Shram Rules:

```
Age BETWEEN 16 AND 59
AND is unorganised worker = True
AND NOT EPFO member AND NOT ESIC member
```

```
flowchart TD
    A[Document Upload] --> B[OCR Extracts Data]
    B --> C{Age >= 60?}
    C -->|Yes| D{BPL Card?}
    C -->|No| E[Not Eligible<br/>Age Requirement]
    D -->|Yes| F[Eligible for IGNOAPS]
    D -->|No| G[Not Eligible<br/>Need BPL First]

    style F fill:#90EE90
    style E fill:#FFB6C1
    style G fill:#FFB6C1
```

4. Data Flow Diagrams

4.1 Voice Query Flow

```
sequenceDiagram
    participant U as User
    participant F as Frontend
    participant B as Backend
    participant WHIS as Speech Service
    participant RE as Rule Engine
    participant RAG as RAG
    participant LLM as LLM

    U->>F: Speak query (Hindi/English)
    F->>B: POST /api/voice-query
    B->>WHIS: Process voice query
    WHIS-->>B: Intent: pension_inquiry

    B->>RE: Check eligibility rules
    RE-->>B: Rules context
```

```

B->>RAG: Search schemes
RAG-->>B: Verified scheme info

B->>LLM: Generate simple explanation
LLM-->>B: Guided response text

B->>F: Response with audio URL
F->>U: Display + TTS output

```

4.2 Document Scan Flow

```

sequenceDiagram
    participant U as User
    participant F as Frontend
    participant B as Backend
    participant OCR as Tesseract OCR
    participant RE as Rule Engine

    U->>F: Upload Aadhaar image
    F->>B: POST /api/scan-document
    B->>OCR: Extract text from image
    OCR-->>B: Raw text + structured data

    B->>RE: Check eligibility
    RE-->>B: Eligibility result

    B-->>F: Extracted data + eligibility
    F->>U: Show results + steps

```

4.3 Eligibility Check Flow

```

flowchart TD
    A[User Query or Document] --> B[Extract Age & Details]

    B --> C{Scheme = IGNOAPS?}
    C -->|Yes| D[Check IGNOAPS Rules]
    C -->|No| E{Scheme = E-Shram?}
    E -->|Yes| F[Check E-Shram Rules]
    E -->|No| G[Unsupported Scheme Error]

    D --> H{Age >= 60?}
    H -->|No| I[Not Eligible<br/>Age < 60]
    H -->|Yes| J{Has BPL?}
    J -->|No| K[Not Eligible<br/>Need BPL]
    J -->|Yes| L[Eligible]

    F --> M{Age 16-59?}
    M -->|No| N[Not Eligible<br/>Age Out of Range]
    M -->|Yes| O{Unorganised Worker?}
    O -->|No| P[Not Eligible<br/>Already Covered]
    O -->|Yes| Q[Eligible]

    I --> R[Return Result]
    K --> R
    L --> R
    N --> R
    P --> R
    Q --> R

```

```

L --> R
N --> R
P --> R
Q --> R

style L fill:#90EE90
style Q fill:#90EE90
style I fill:#FFB6C1
style K fill:#FFB6C1
style N fill:#FFB6C1
style P fill:#FFB6C1

```

5. Technology Stack Table

Layer	Technology	Version	Purpose
Frontend			
UI Framework	React	18.2.0	Component-based UI
Language	TypeScript	4.9.5	Type safety
Build Tool	React Scripts	5.0.1	Development/build
Backend			
Framework	FastAPI	0.111.0	REST API
Server	Uvicorn	0.30.1	ASGI server
Validation	Pydantic	2.7.1	Data validation
HTTP Client	httpx	0.27.0	Async HTTP
AI/ML			
Speech-to-Text	OpenAI Whisper	Latest	Voice transcription
OCR	Tesseract	Latest	Document text extraction
OCR Python	pytesseract	0.3.10	Tesseract wrapper
Image Processing	Pillow	10.3.0	Image handling

Layer	Technology	Version	Purpose
Data			
Knowledge Base	Custom RAG	-	Government schemes

6. API Request/Response Examples

6.1 Voice Query

Request :

```
POST /api/voice-query
{
  "query": "pension inquiry",
  "language": "auto"
}
```

Response :

```
{
  "success": true,
  "intent": "pension inquiry",
  "confidence": 0.9,
  "response_text": "pension inquiry",
  "audio": {
    "audio_url": "/api/audio/8321.mp3",
    "text": "pension inquiry",
    "duration_seconds": 6.5,
    "language": "hi-IN"
  },
  "suggested_actions": ["scan document", "check eligibility"],
  "language_detected": "mixed"
}
```

6.2 Document Scan

Request :

```
POST /api/document-scan
{
  "document_url": "https://example.com/document.pdf",
  "language": "auto"
}
```

Response :

```
{
  "success": true
}
```

```

{
  "extracted_data": {
    "name": "Ramkali Devi",
    "dob": "12/03/1954",
    "age": 72,
    "gender": "Female",
    "has_bpl": true,
    "raw_text_preview": "[DEMO MODE] Aadhaar card for Ramkali Devi ..."
  },
  "document_type": "aadhaar_card",
  "confidence": 0.95,
  "message": "Document processed successfully"
}

```

6.3 Eligibility Check

Request :

```

{
  "name": "Ramkali Devi",
  "dob": "12/03/1954",
  "age": 72,
  "gender": "Female",
  "has_bpl": true
}

```

Response :

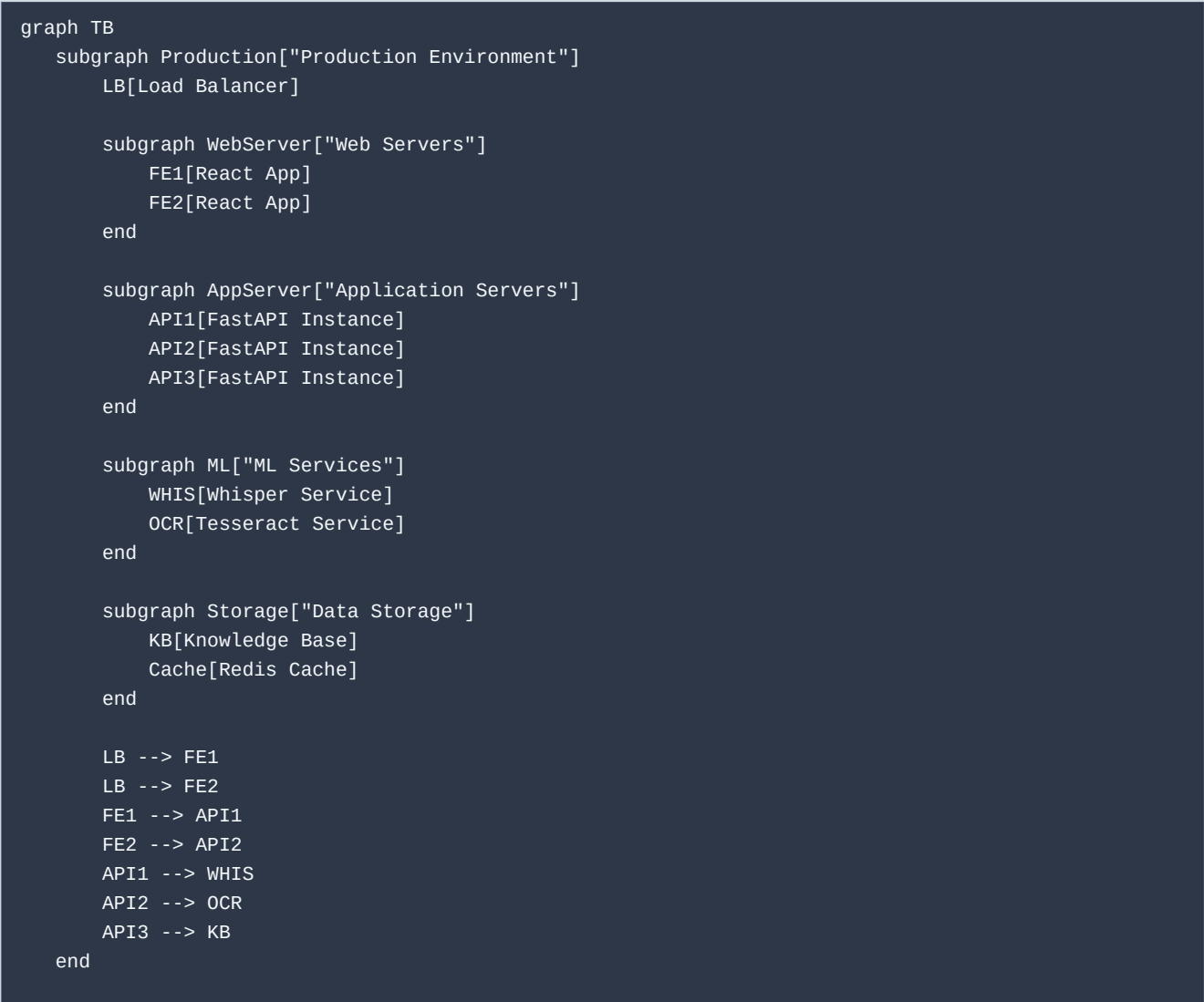
```

{
  "success": true,
  "eligible": true,
  "scheme": "IGNOAPS",
  "verdict": "Eligible for IGNOAPS Pension",
  "reasons": [
    "✓ Age is 72 years - meets the minimum age requirement of 60.",
    "✓ BPL card status confirmed."
  ],
  "steps": [
    "Step 1 - Gather documents: Aadhaar card, BPL ration card, bank passbook, ..."
    "Step 2 - Visit your local Gram Panchayat / Ward Office, ..."
    "Step 3 - Fill the form with the help of the ward officer, ..."
    "Step 4 - Submit the form at the office and collect the acknowledgement slip."
  ],
  "guided_response": {
    "text": "You are eligible for IGNOAPS Pension. Please follow the steps to apply.",
    "audio": {
      "audio_url": "/api/audio/4521.mp3",
      "language": "hi-IN"
    }
  }
}

```

7. Implementation Considerations

7.1 Deployment Architecture



7.2 Scalability Recommendations

Component	Current	Recommended for Scale
Frontend	Static React	CDN + Edge caching
Backend	Single FastAPI	Multiple instances behind load balancer
OCR	Local Tesseract	Cloud Vision API (Google/AWS)
Speech	Mock/Local	OpenAI Whisper API

Component	Current	Recommended for Scale
Knowledge	In-memory	Vector database (Pinecone/Weaviate)

7.3 Security Considerations

- 1. **PII Handling:** Documents are processed and discarded; not stored
- 2. **API Security:** CORS restricted to frontend origin
- 3. **Input Validation:** Pydantic models validate all inputs
- 4. **No Training Data:** User data never used for model training

7.4 Accessibility Compliance

- WCAG 2.1 AA target compliance
- Screen reader tested components
- Minimum 4.5:1 contrast ratio
- Focus indicators for keyboard navigation
- Alt text for all images
- ARIA labels for interactive elements

8. Getting Started

Prerequisites

- Node.js 16+ and npm
- Python 3.8+
- Tesseract OCR (optional for production)

Running the Application

Backend:

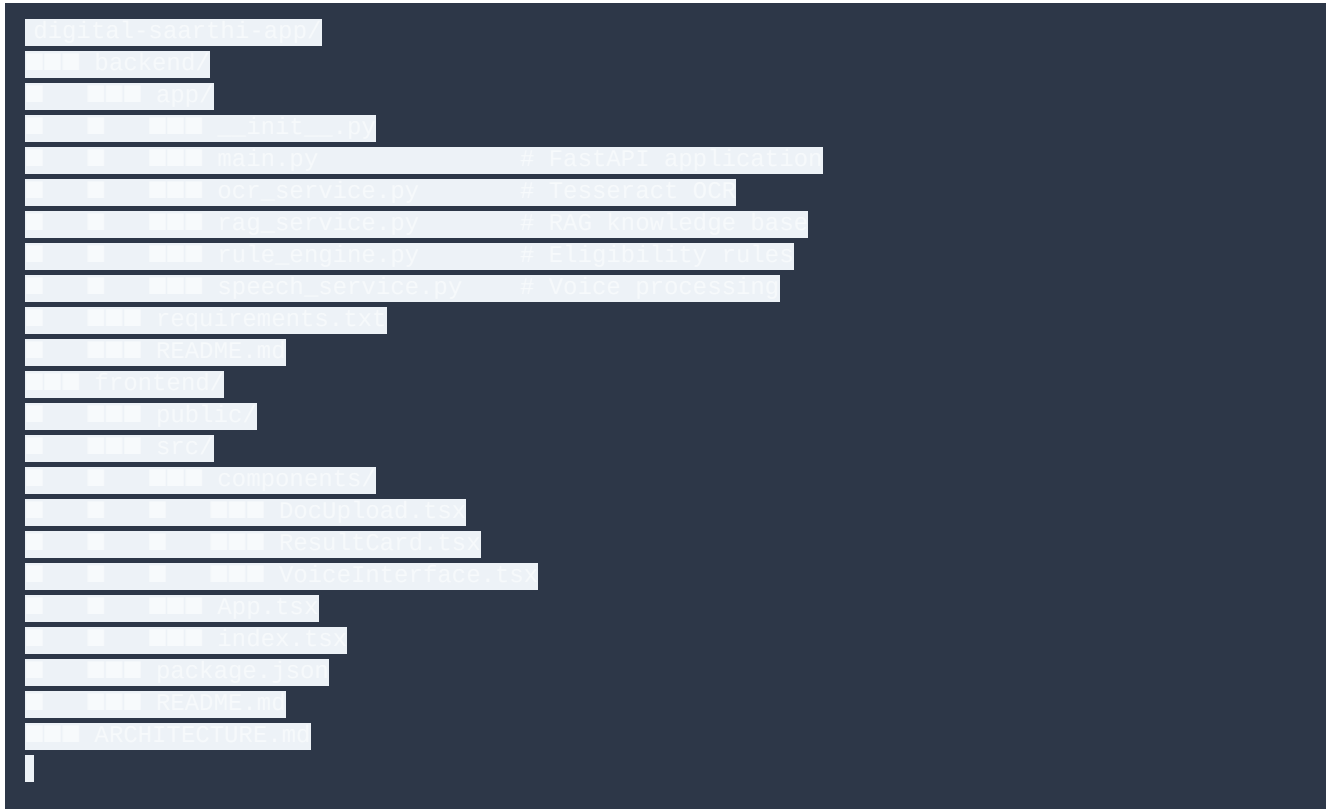
```
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Frontend:

```
cd frontend
npm install
npm start
```

Access: - Frontend: <http://localhost:3000> - Backend API: <http://localhost:8000> - API Docs: <http://localhost:8000/docs>

9. File Structure



10. Summary

Digital Saarthi implements a **safety-first architecture** where:

1. **Rule Engine** makes all eligibility decisions (no hallucinations)
2. **RAG** provides verified government scheme information
3. **LLM** only explains, never decides
4. **Accessibility** is built into every user interaction
5. **Privacy** is respected by not retaining user data

The system successfully bridges the digital divide by making government services accessible to elderly and low-literacy users through voice interfaces, simple visual guidance, and verified step-by-step action plans.